

Concept of Weak / Strong Entities, Participation (Total / Partial), Constraints

Strong Entities vs. Weak Entities

1. Strong Entity:

- Can be uniquely identified by its own attributes.
- Has a **primary key** that uniquely identifies each instance of the entity.
- Example: A Student entity with a primary key StudentID.

2. Weak Entity:

- Cannot be uniquely identified by its own attributes alone.
- Depends on a **strong entity** for its existence (this is called an **identifying relationship**).
- Requires a **discriminator key** (partial key) in combination with the primary key of the strong entity to uniquely identify its instances.
- Example: A Dependent entity (weak) related to an Employee entity (strong). The Dependent entity might have a partial key like DependentName, but it needs the EmployeeID (from the Employee entity) to uniquely identify a dependent.

Concept of Discriminator Key / Partial Key / Partial Discriminator Key

The terms **discriminator key** and **partial discriminator key** and **Partial Key** are often used interchangeably in the context of **weak entities** in ER diagrams. But it is preferred to use **discriminator key** or **partial discriminator key** instead of just using the **Partial Key**.

Definitions and relation with other keys:

1. Primary Key:

- Uniquely identifies an instance of a strong entity.
- Example: StudentID in the Student entity.

2. Foreign Key:

- A field in one table that refers to the primary key of another table.
- In the context of weak entities, the foreign key is the primary key of the strong entity that the weak entity depends on.
- Example: EmployeeID in the Dependent entity (weak) refers to the EmployeeID in the Employee entity (strong).

3. Super Key:

- A set of attributes that can uniquely identify an entity.
- A primary key is a minimal super key.

4. Candidate Key:

- A minimal super key that can uniquely identify an entity.
- There can be multiple candidate keys, but only one is chosen as the primary key.

5. Discriminator Key or Partial Discriminator Key:

- Works in combination with the foreign key (primary key of the strong entity) to uniquely identify a weak entity.
- Example: DependentName in the Dependent entity, combined with EmployeeID.

Example

Let’s consider a scenario with a **strong entity** (Employee) and a **weak entity** (Dependent):

Entities and Attributes

- **Employee (Strong Entity):**
 - Attributes: EmployeeID (Primary Key), Name, Department.
- **Dependent (Weak Entity):**
 - Attributes: DependentName (Partial Key), Relationship, DateOfBirth.
 - Foreign Key: EmployeeID (refers to the Employee entity).

Identifying Relationship

- The Dependent entity depends on the Employee entity for its existence.
- The EmployeeID (foreign key) from the Employee entity is combined with the DependentName (partial key) to uniquely identify a dependent.

ER Diagram Representation

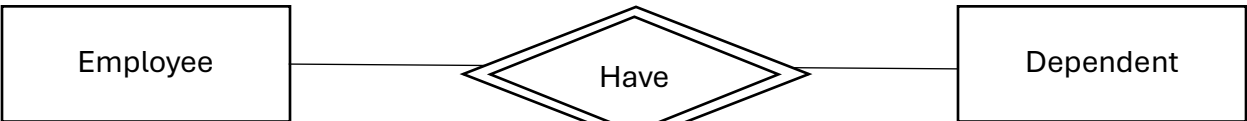
- In the ER diagram:
 - The Employee entity has a primary key EmployeeID.
 - The Dependent entity has a partial key DependentName (underlined with a dashed line).
 - The relationship between Employee and Dependent is an **identifying relationship** (represented by a double diamond in Chen notation).

Table Representation

- **Employee Table:**

EmployeeID (PK)	Name	Department
101	John Doe	HR
102	Jane Smith	IT
- **Dependent Table:**

EmployeeID (FK)	DependentName (Partial Key)	Relationship	DateOfBirth
101	Alice	Daughter	2010-05-12
101	Bob	Son	2012-08-23
102	Charlie	Son	2015-03-17
- Here, the combination of EmployeeID and DependentName uniquely identifies each dependent.



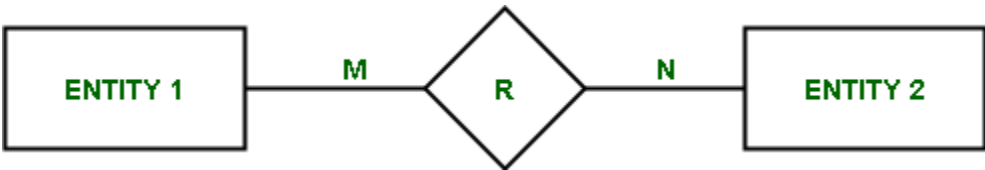
Structural Constraints of Relationships in ER Model

Structural constraints, within the context of Entity-Relationship (ER) modeling, specify and determine how the entities take part in the relationships, and this gives an outline of how the interactions between the entities can be designed in a database. Two primary types of constraints are **cardinality** (specifying the number of instances in a relationship) and **participation** (whether

all or some entity instances are involved in the relationship). Cardinality is defined as one-to-one, one-to-many, or many-to-many, while participation can be total or partial.

This forms the constraints that need to be used in defining an accurate and efficient database schema since the introduction of the interaction rules of entities ensures that the correctness of the data model reflects in the real situation. The knowledge of these constraints is very essential for the design of strong and functional databases.

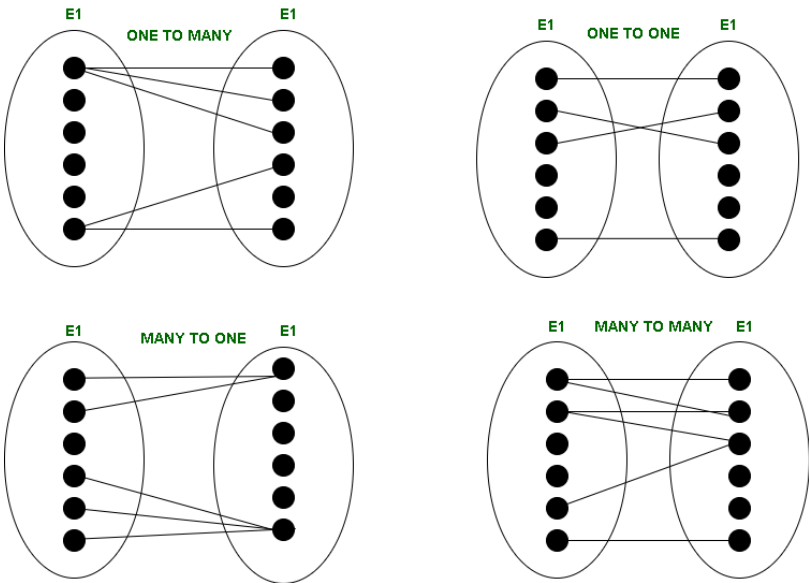
To understand Structural Constraints, we must take a look at Cardinality Ratios and Participation Constraints. **Cardinality Ratios of relationships:** The entities are denoted by rectangle and relationships by diamond.



There are numbers (represented by M and N) written above the lines which connect relationships and entities. These are called cardinality ratios. These represent the maximum number of entities that can be associated with each other through relationship, R.

Types of Cardinalities in ER-Model

There can be 4 types of cardinalities



1. **One-to-one (1:1)** – When one entity in each entity set takes part at most once in the relationship, the cardinality is one-to-one.
2. **One-to-many (1: N)** – If entities in the first entity set take part in the relationship set at most once and entities in the second entity set take part many times (at least twice), the cardinality is said to be one-to-many.
3. **Many-to-one (N:1)** – If entities in the first entity set take part in the relationship set many times (at least twice), while entities in the second entity set take part at most once, the cardinality is said to be many-to-one.
4. **Many-to-many (N: N)** – The cardinality is said to be many to many if entities in both the entity sets take part many times (at least twice) in the relationship set.

Concept of Total Participation & Partial Participation using Cardinality

Participation constraints tell us that participation in a relationship can either be total or partial.

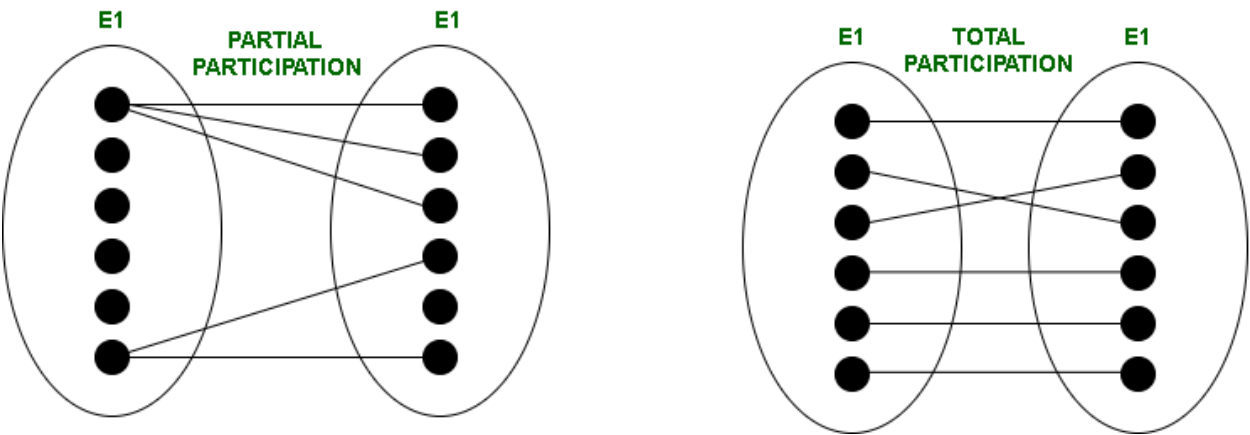
Total Participation

When each entity in an entity set participates in a relation, it is called Total Participation. In ER modeling, total participation exists when the instances of the entity must necessarily participate in

at least one relationship instance. For example, in a university system, where every student must be enrolled in at least one course, total participation exists in the relationship between “Student” and “Course”. Similarly, if each professor teaches at least one course, then the relationship between “Professor” and “Course” also displays total participation. The notion here is that no entity can exist without being involved in the relationship.

Partial Participation

When all entities in the given entity set do not participate in a relation, it is called Partial Participation. Contrary to full participation, partial participation allows some components of a particular entity to be present without being part of any relation. Some entities may or may not be part of the relationship, and, in case of a single line in the ER diagram, that is indicated. For instance, in the university system, if some professors do not teach any courses, then their participation would be partial in the “Teaches” relationship.



Difference b/w Associative Entity & Table Created for a relation: How to know which one should be created?

The distinction between an **associative entity** and a **table created by a relationship** can be subtle, but it is important in database design. Both concepts are used to resolve many-to-many (M:N) relationships, but they differ in their representation and usage. Let’s break it down in the context of **Chen notation** and **Crow’s Foot notation**, and clarify how to decide which one to use.

1. Associative Entity

An **associative entity** is an entity that is used to resolve a many-to-many relationship between two or more entities. It has its own identity and may have additional attributes that describe the relationship.

Characteristics:

- Represents a **many-to-many relationship**.
- Has its own **primary key** (which may be a composite key made up of foreign keys from the related entities).
- Can have **additional attributes** that describe the relationship.
- Treated as an **entity** in the ER diagram.

When to Use:

- When the relationship has **attributes** that need to be stored.
- When the relationship itself has an **independent meaning** or identity in the database.

2. Table Created by a Relationship

A **table created by a relationship** is a junction table that resolves a many-to-many relationship but does not have its own identity or additional attributes. It simply stores the foreign keys of the related entities.

Characteristics:

- Represents a **many-to-many relationship**.
- Does not have its own **primary key** (it uses a composite key made up of foreign keys from the related entities).
- Does not have **additional attributes**.
- Treated as a **relationship** in the ER diagram.

When to Use:

- When the relationship does **not have attributes**.
- When the relationship is purely a **junction** between two entities.

Difference in Chen Notation

Associative Entity:

- Represented as a **rectangle** (entity) with a **diamond** (relationship) inside it.
- Connected to the related entities with **solid lines**.
- Example: In a Student and Course relationship, an associative entity Enrollment might have attributes like EnrollmentDate and Grade.

Table Created by a Relationship:

- Represented as a **diamond** (relationship) connected to the related entities with **solid lines**.
- No additional attributes are shown.
- Example: In a Student and Course relationship, a simple Enrolls relationship might not have any attributes.

Difference in Crow’s Foot Notation

Associative Entity:

- Represented as a **rectangle** (entity) with a **foreign key relationship** to the related entities.
- The associative entity has its own **primary key** and **additional attributes**.
- Example: In a Student and Course relationship, an associative entity Enrollment might have attributes like EnrollmentDate and Grade.

Table Created by a Relationship:

- Represented as a **junction table** with **foreign keys** referencing the related entities.
- No additional attributes are shown.
- Example: In a Student and Course relationship, a simple Enrolls table might only store StudentID and CourseID.

How to Decide: Associative Entity vs. Table Created by a Relationship

1. **Check for Attributes:**

- If the relationship has **attributes**, use an **associative entity**.
- Example: In a Student and Course relationship, if you need to store EnrollmentDate and Grade, use an associative entity.

2. **Check for Independent Meaning:**

- If the relationship has an **independent meaning** or identity, use an **associative entity**.
- Example: In a Supplier, Part, and Project relationship, if you need to track QuantitySupplied and SupplyDate, use an associative entity.

3. **Check for Simplicity:**

- If the relationship is **simple** and does not have attributes, use a **table created by a relationship**.
- Example: In a Student and Course relationship, if you only need to track which students are enrolled in which courses, use a simple junction table.

Example 1: Associative Entity

Scenario:

- A Student can enroll in many Courses, and a Course can have many Students. The enrollment has attributes like EnrollmentDate and Grade.

Chen Notation:

- Student and Course are connected to an associative entity Enrollment (rectangle with a diamond inside).
- Enrollment has attributes: EnrollmentDate and Grade.

Crow’s Foot Notation:

- Enrollment is represented as a rectangle with foreign keys to Student and Course.
- Enrollment has its own primary key (composite key: StudentID + CourseID) and attributes.

Tables:

- **Student:**

StudentID (PK)	Name
101	John Doe
102	Jane Smith
- **Course:**

CourseID (PK)	CourseName
C101	DBMS
C102	Algorithms
- **Enrollment:**

StudentID (FK)	CourseID (FK)	EnrollmentDate	Grade
101	C101	2023-09-01	A
101	C102	2023-09-01	B
102	C101	2023-09-01	A

Example 2: Table Created by a Relationship

Scenario:

- A Student can enroll in many Courses, and a Course can have many Students. The relationship has no additional attributes.

Chen Notation:

- Student and Course are connected by a Enrolls relationship (diamond).

Crow’s Foot Notation:

- A junction table Enrolls is created with foreign keys to Student and Course.

Tables:

- **Student:**

StudentID (PK)	Name
101	John Doe
102	Jane Smith
- **Course:**

CourseID (PK)	CourseName
C101	DBMS
C102	Algorithms
- **Enrolls:**

StudentID (FK)	CourseID (FK)
101	C101
101	C102
102	C101

Summary

- Use an **associative entity** when the relationship has **attributes** or **independent meaning**.
- Use a **table created by a relationship** when the relationship is **simple** and has no attributes.